
LeanVIPR: A Formally Verified Certificate Checker for Exact Mixed-Integer Programming

Cruise Song

Georgia Institute of Technology
Atlanta, GA 30332

Hai Duong

George Mason University
Fairfax, VA 22030

ThanhVu Nguyen

George Mason University
Fairfax, VA 22030

Vijay Ganesh

Georgia Institute of Technology
Atlanta, GA 30332

Abstract

To establish trust in mixed-integer programming (MIP), modern solvers, e.g., SCIP, can emit VIPR certificates for infeasibility and objective-range results. These certificates make solver results independently checkable and capture the proof steps produced by a practical, general-purpose MIP solver. However, the VIPR reference checker is an unverified C++ program, so an implementation error could cause it to accept an invalid certificate.

We present LEANVIPR, a VIPR checker implemented and formally verified in LEAN 4. We formalize the semantics and soundness of the VIPR proof system, including linear combination, integer rounding, assumptions, and branch closure, and prove that LEANVIPR is sound and terminating. Consequently, acceptance establishes either that the original MIP is infeasible or that the listed solutions are feasible and the reported objective range is valid. To improve checking performance for infeasibility certificates, we also support sound backward slicing: acceptance of a sliced certificate establishes infeasibility of the original MIP.

1 Introduction

Modern mixed-integer programming (MIP) solvers combine branch-and-bound with numerous inference and numerical routines. Exact rational solvers avoid unsound conclusions caused by floating-point error, but remain substantial software systems [Cook et al., 2013, Eifler and Gleixner, 2023]. An incorrect infeasibility or objective-range result can invalidate every downstream conclusion that depends on it. Certification reduces this risk: instead of trusting the solver, we ask it to emit a machine-readable proof and validate that proof with an independent checker.

The Verifying Integer Programming Results (VIPR) format provides such certificates for general MIP reasoning. A VIPR certificate lists constraints in order, with each derived constraint citing earlier constraints and the rule used to derive it. Its rules cover exact linear combinations, integer rounding, and branch-and-bound case splits, and exact SCIP can produce the certificates [Cheung et al., 2017, Eifler and Gleixner, 2023]. This makes VIPR a practical interface between a full MIP solver and a smaller independent checker.

The original VIPR work defines the format, justifies its rules mathematically, integrates certificate production into exact SCIP, and implements the C++ reference checker VIPRCHK [Cheung et al., 2017]. Checking with VIPRCHK reduces the amount of trusted code relative to trusting SCIP, but neither the checker implementation nor the soundness arguments for its rules are machine-checked.

Consequently, an error in arithmetic, rounding, or branch handling could cause VIPRCHK to accept an invalid certificate.

The closest prior work formalizes VIPR validity in WHY3 as a formula in linear integer-real arithmetic [Wood et al., 2025, Filiâtre and Paskevich, 2013]. Runtime checking, however, relies on a separate C++ translator that emits SMT-LIB and on CVC5, leaving those components outside the formal proof [Barrett et al., 2010, Barbosa et al., 2022].

In this work, we implement LEANVIPR, an executable VIPR checker in LEAN 4¹. We connect its acceptance of a certificate to a machine-checked proof of the claimed MIP result: infeasibility, or feasibility of the listed solutions together with validity of the reported objective range. Note that we do not verify the solver or the full VIPR toolchain. Following formally verified certificate checkers for SAT and pseudo-Boolean reasoning [Szeider, 2026a,b], we formalize both LEANVIPR’s executable replay and its soundness proof in LEAN [Moura and Ullrich, 2021].²

Given a parsed VIPR certificate, LEANVIPR uses exact rational numbers to replay its derivation steps in order while tracking the branch assumptions behind each constraint. During replay, `asm` opens a branch assumption, `lin` checks a signed linear combination, `rnd` checks integer rounding, and `uns` joins complementary branches and removes their assumptions. If a rule check fails, LEANVIPR rejects the certificate. For an infeasibility certificate, LEANVIPR accepts only if the final constraint is impossible, such as $0 \leq -1$, and has no open branch assumptions. For an objective-range certificate, it additionally checks the listed solution witnesses and the closed derived bound on the objective.

We prove in LEAN that each rule check preserves validity for real assignments satisfying the declared integrality conditions and current branch assumptions. Applying these proofs step by step, we show that each derived constraint is implied by the original MIP constraints and the branch assumptions on which it depends. This invariant yields our main soundness theorem: if LEANVIPR accepts an infeasibility certificate, no real assignment satisfying the declared integrality conditions can satisfy the original MIP constraints; if it accepts an objective-range certificate, every listed solution is feasible and every optimal objective value lies in the reported range. The theorem applies directly to the executable checker’s Boolean result, connecting successful replay to the claimed MIP result. To reduce replay cost for infeasibility certificates, LEANVIPR retains only the constraints and derivation steps needed for the final contradiction. We prove that acceptance of this sliced certificate soundly refutes the original MIP.

Our contributions are:

- A formal model in LEAN of linear constraints and the VIPR rules for linear combination, integer rounding, branch assumptions, and unsplitting, together with proofs that these rules are sound.
- A sound and terminating executable VIPR checker for infeasibility and objective-range certificates.
- A backward-slicing optimization for infeasibility certificates, with a proof that acceptance of the slice establishes infeasibility of the original MIP.

2 Background and Example

2.1 VIPR Certificates

A VIPR certificate contains the variables and their integer declarations (`VAR`, `INT`), the objective and original constraints (`OBJ`, `CON`), the claimed result (`RTP`), and a flat derivation (`DER`) [Cheung et al., 2017, Wood et al., 2025]. LEANVIPR supports the VIPR v1.0 rules together with the v1.1 `lin weak` extension used by SCIP-generated certificates. Original and derived rows share one index space, so a derivation reason can refer directly to any earlier row. Four reasons are central to an infeasibility proof:

- `asm` introduces a temporary branch bound;
- `lin` states a row dominated by a suitable signed linear combination of earlier rows;

¹Hereafter, we refer to LEAN 4 simply as LEAN.

²The source code is available at <https://anonymous.4open.science/r/LeanVIPR/>

```

VER 1.0
VAR 3 t_y t_x t_z
INT 3 0 1 2
OBJ min 0
CON 8 6
C0 E 23 3 0 10 1 6 2 15
C1 G 1 3 0 1 1 1 2 1
B2 G 0 1 0 1
B3 L 2 1 0 1
B4 G 0 1 1 1
B5 L 3 1 1 1
B6 G 0 1 2 1
B7 L 1 1 2 1
RTP infeas
SOL 0
DER 32
DualBound_8 G 0 OBJ { lin 0 } -1
A9 L 0 1 0 1 { asm } -1
A10 G 1 1 0 1 { asm } -1
ACT_L12 ... { lin 3 0 -1 10 10 4 6 } -1
ACT_R14 ... { rnd 1 13 1 } -1
... (branch y>=1) ...
ActivityConflict27 L -1 0 { lin 4 ... } -1
... (branch y<=0) ...
ActivityConflict38 L -2 0 { lin 4 ... } -1
UnsplitNode1_39 G 1 0 { uns 27 10 38 9 } -1

```

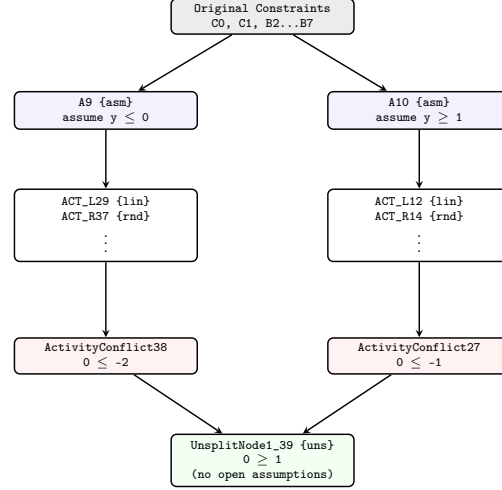


Fig. 1: The real exact-SCIP certificate (left, truncated) alongside its branch-and-bound derivation DAG (right, demonstration only) for the infeasible problem in Eq. 1. The tree splits on integer variable y , tightening relaxations via **lin** and **rnd** steps until each branch reaches an absurdity. The **uns** step merges the branches and discharges the assumptions.

- **rnd** performs such a combination and rounds its bound after checking that the resulting form is integer-valued, as in exact cutting-plane reasoning [Eifler and Gleixner, 2024]; and
- **uns** combines two branches justified by complementary integer bounds and discharges those bounds.

A row is therefore more than a constraint: during replay it also carries the set of branch assumptions on which it depends. An infeasibility certificate closes only when its final row is an absurdity, such as $0 \leq -1$, and its assumption set is empty.

2.2 A Feasibility Checking Example

Consider the integer-infeasible feasibility checking problem in Eq. 1 as our running example:

$$\begin{aligned}
 \min \quad & 0 \\
 \text{s.t.} \quad & 6x + 10y + 15z = 23 \\
 & x + y + z \geq 1 \\
 & 0 \leq x \leq 3, \quad 0 \leq y \leq 2, \quad 0 \leq z \leq 1 \\
 & x, y, z \in \mathbb{Z}
 \end{aligned} \tag{1}$$

The linear relaxation of Eq. 1 is feasible at $(x, y, z) = (1/2, 1/2, 1)$. This point is not a feasible MIP solution because $y = 1/2$ violates the integrality requirement. Since an integer value of y cannot lie strictly between 0 and 1, every feasible MIP assignment must satisfy either $y \leq 0$ or $y \geq 1$. Exact SCIP exploits this exhaustive integer split and derives a contradiction in both branches. Fig. 1 pairs an excerpt of the resulting certificate with its branch-and-bound structure.

We next trace how LEANVIPR replays this flat certificate and verifies the branch argument in four steps.

Translate the certificate. LEANVIPR processes the flat derivation list shown in Fig. 1. Function `derToSStep` converts each parsed **DER** record into an **SStep**, consisting of a sensed row **SLe** and a reason **SReason**, such as **asm**, **lin**, **rnd**, or **uns**. Function `initialPool` initializes the array-backed `CheckerPool` with the eight original **CON** rows converted by `conToSLe`.

Track branch assumptions. Each `CheckerEntry` contains a row and the finite set **SAsm** of assumptions on which it depends. Rows **A9** and **A10** introduce the complementary cases $y \leq 0$ and

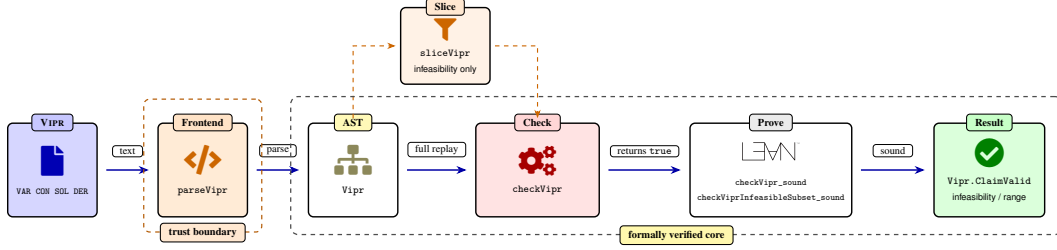


Fig. 2: LEANVIPR’s executable checker and soundness chain. The blue path replays the complete certificate. The dashed orange path is optional untrusted backward slicing for infeasibility certificates; checked subset conditions transport its result to the original MIP.

$y \geq 1$. Function `entryOf` records an `asm` step’s own bound and uses `collectReferencedAsms` so that `lin` and `rnd` steps inherit the assumptions of the rows they reference.

Check the branch derivations. For example, `ACT_L12` cites the exact combination

$$-1 \cdot C_0 + 10 \cdot A_{10} + 6 \cdot B_4.$$

`resolveRefs` retrieves rows 0, 10, and 4, and `checkLin` verifies that the signed multipliers are suitable for their senses, computes the sparse rational combination, and checks that it dominates the stated row. The theorem `stepValidB_sound` turns a successful Boolean step check into the declarative relation `Vipr.StepValid`; `Vipr.replay_sound` then proves that the replayed entries are semantically sound over real assignments. A `rnd` step is checked by `checkRnd`, which additionally uses `formIntB` to establish that the combined form is integer-valued before rounding its bound. Repeating these checks derives `ActivityConflict27`, $0 \leq -1$, under `A10`, and `ActivityConflict38`, $0 \leq -2$, under `A9`.

Discharge the split and certify infeasibility. For the final reason `uns 27 10 38 9`, `stepValidB` uses `domSLe` to check that rows 27 and 38 each dominate the stated conclusion and `unsBoundsB` to check that rows 10 and 9 are complementary integer bounds. Function `entryOf` then removes those branch bounds from the corresponding assumption sets. The resulting row $0 \geq 1$ is absurd and has no open assumptions. The production checker therefore returns `checkVipr cert = true`. Its main theorem, `checkVipr_sound`, yields `Vipr.ClaimValid cert`; because this certificate has `RTP infeas`, this states that no real assignment satisfying the declared integrality conditions can satisfy all constraints in Eq. 1.

3 The LEANVIPR Approach

Fig. 2 separates three parts of LEANVIPR: the textual frontend, the executable checker, and the proof that Boolean acceptance implies the claimed MIP result. The main soundness theorem begins with a parsed `Vipr` value; correctness of `parseVipr` with respect to source text is therefore a separate frontend concern.

Certificate Semantics A sparse linear form is represented as a list of variable-index and rational-coefficient pairs. An `SLe` row pairs such a form and a rational right-hand side with a sense: `L` for $\leq$, `G` for $\geq$, or `E` for equality. The function `conSLes` converts the original `CON` records into sensed rows, while `viprSteps` applies `derToSStep` to every `DER` record.

This translation preserves the certificate’s shared index space, so each derivation can resolve its references directly against the replay pool. Although the representation uses exact rationals, its soundness is stated for real valuations whose declared integer variables have integer values.

Executable Replay The production checker stores replay entries in an array `CheckerPool`. Each entry is a pair (c, A) consisting of a row c and the finite set A of branch assumptions on which it depends. The original constraints initialize the array with empty assumption sets.

Alg. 1 mirrors the implementation. The Boolean `stepValidB` checks that all references point backward and validates the selected `asm`, `lin`, `rnd`, `uns`, or `sol` rule. Validation and replay are fused:

Alg. 1: Production checker `checkVipr`.

Input: Parsed certificate v

```
1 if  $\neg \text{solSectionB}(v)$  then
2   return false // Reject invalid solution data
3  $P \leftarrow \text{initialPool}(v)$  // Array of original rows
4 foreach  $s \in \text{viprSteps}(v)$  do
5   if  $\neg \text{stepValidB}(v, P, s)$  then
6     return false // Reject the first invalid step
7    $P.\text{push}(\text{entryOf}(P, s))$  // Append row and assumptions
8 return  $\text{finalValidB}(v, P.\text{back}?)$ 
```

after a successful check, `entryOf` computes the new assumption set and appends the entry to the array. Because `replaySteps?` recurses structurally over the finite step list, the checker is total and terminating.

The final condition depends on `RTP`. For `infeas`, `finalValidB` requires an assumption-free absurd row. For a minimization range, the derivation supplies the lower bound while a checked feasible `SOL` witness supplies the upper bound; maximization reverses these roles. Thus range acceptance checks both proof replay and the listed primal solutions, rather than treating the absence of a contradiction as evidence of feasibility.

Declarative Proof System and Soundness The semantic proof system is deliberately separate from the executable array implementation. `Vipr.StepValid` specifies the mathematical validity of one rule, `Vipr.DerivationValid` lifts it to a derivation, and `Vipr.ProofValid` combines replay validity, solution validity, and the appropriate final-row condition. For example, a `lin` step is valid when every reference points to an earlier entry and a sign-compatible exact rational combination of the referenced rows dominates the stated row; replay assigns the new entry the union of their open assumptions.

The theorem `stepValidB_sound` maps successful Boolean checking of one array-backed step to `Vipr.StepValid`. The replay refinement theorem `replaySteps_refines` lifts this correspondence to the complete array replay, after which `checkVipr_refines` establishes `Vipr.ProofValid`. Independently, `Vipr.proofSystem_sound` proves the soundness of the declarative VIPR system. Their composition is the main implementation theorem:

$$\text{checkVipr } v = \text{true} \implies \text{Vipr.ClaimValid } v.$$

This is the statement of `checkVipr_sound`. For `RTP infeas`, `Vipr.ClaimValid` excludes every real feasible assignment satisfying the integrality declarations. For `RTP range`, it states that the listed solutions are valid and that every optimal objective value lies in the reported interval.

Backward Slicing For infeasibility certificates, `sliceVipr` follows references backward from the final contradiction, retains only reachable `CON` and `DER` rows, and remaps their indices. The slicing code is an untrusted optimization. The theorem `checkViprInfeasibleSubset_sound` transports infeasibility only when the concrete slice has the same integer variables, every retained original row occurs in the original system, and `checkVipr` accepts the slice. A generated reconstruction artifact checks these premises for the particular certificate. Consequently, an incorrect slice is rejected rather than changing the soundness claim. Range certificates currently use complete replay because their solution witnesses and objective bounds refer to the original problem.

Evaluation We evaluate the native LEANVIPR executable on 26 exact-SCIP certificates: 20 infeasibility certificates from the ALU suite used in exact-MIP experiments [Eifler and Gleixner, 2023] and six objective-range certificates from ALU and MIPLIB 3.0. Tab. 1 reports native execution on an Apple M3 Pro. On the same 20 infeasibility certificates, slicing reduces replay from 124,382 to 33,568 steps—a 73% reduction—and aggregate wall time from 8.49 to 3.44 seconds, a $2.5\times$ speedup. All 46 measured invocations pass: complete and sliced replay for each infeasibility certificate, and complete replay for each range certificate. Soundness follows from the theorems above rather than from the execution environment.

Tab. 1: Native LEANVIPR replay. The full and sliced infeasibility rows use the same 20 certificates.

Certificate mode	Certificates	Replayed steps	Wall time (s)
Infeasibility, full	20	124,382	8.49
Infeasibility, sliced	20	33,568	3.44
Objective range, full	6	95,145	28.41

Trust Base At the theorem level, the trusted base is the standard LEAN kernel, and the development introduces no additional axioms. For a particular source file, native execution additionally trusts the VIPR parser and the LEAN compiler and runtime. This is a common performance–trust tradeoff for verified decision procedures using native evaluation, including LRATCATCHER and PBLEAN [Szeider, 2026a,b].

We also experimented with proof artifacts whose concrete Boolean equality is discharged by kernel reduction using `decide +kernel`, removing the LEAN compiler and runtime from the trusted base. This mode did not complete the full suite under a five-minute per-instance limit, so scalable kernel-only replay remains future work.

4 Related Work

MIP certificate checking. Cheung et al. [2017] introduced VIPR, certificate support in exact SCIP, and the C++ checker VIPRCHK. Closest to our work, Wood et al. [2025] formalize VIPR validity in WHY3 [Filliâtre and Paskevich, 2013] and relate it to a ground linear integer-real arithmetic formula, while a separate C++ frontend emits SMT-LIB for CVC5 [Barrett et al., 2010, Barbosa et al., 2022]. We instead implement certificate replay and prove the checker sound directly in LEAN, without relying on an external SMT solver in the soundness argument.

Formally verified certificate checking. LRATCATCHER checks SAT refutations in LEAN by reflection [Szeider, 2026a]. For pseudo-Boolean reasoning, PBLEAN reflectively checks VERIPB certificates in LEAN [Szeider, 2026b], while CAKEPB obtains a verified standalone checker through HOL4 and CAKEML [Gocht et al., 2024]. These systems provide formally verified checking for SAT and pseudo-Boolean proof systems. LEANVIPR extends this line of work to certificates produced by a general MIP solver, including exact rational constraints, continuous and integer variables, integrality-based rounding, and explicit discharge of branch assumptions.

5 Conclusion and Future Work

We presented LEANVIPR, a formally verified LEAN checker for VIPR infeasibility and objective-range certificates. LEANVIPR checks linear combination, rounding, assumptions, and unsplitting, and proves that these checks preserve validity. Its executable Boolean checker is linked by a LEAN theorem to the claimed MIP result, without relying on the solver, the reference checker, or an external SMT solver for the soundness argument.

Future work will investigate completeness of the VIPR proof system, formalize the checker’s algorithmic complexity, verify the VIPR parser to remove it from the trusted base, and scale kernel-only replay and evaluation to larger certificates.

AI assistance disclosure. An LLM-based coding assistant was used during development to assist with LEAN implementation and proof engineering, benchmark scripting, and manuscript editing. The assistant is not part of LEANVIPR or its trusted base. All resulting code and proofs were reviewed by the authors, the formal development was checked by LEAN, and the authors take responsibility for the paper and artifact.

References

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. cvc5: A versatile and industrial-strength smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442. Springer, 2022.

- Clark Barrett, Aaron Stump, Cesare Tinelli, et al. The smt-lib standard: Version 2.0. In *Proceedings of the 8th international workshop on satisfiability modulo theories (Edinburgh, UK)*, volume 13, page 14, 2010.
- Kevin KH Cheung, Ambros Gleixner, and Daniel E Steffy. Verifying integer programming results. In *International Conference on Integer Programming and Combinatorial Optimization*, pages 148–160. Springer, 2017.
- William Cook, Thorsten Koch, Daniel E Steffy, and Kati Wolter. A hybrid branch-and-bound approach for exact rational mixed-integer programming. *Mathematical Programming Computation*, 5(3): 305–344, 2013.
- Leon Eifler and Ambros Gleixner. A computational status update for exact rational mixed integer programming. *Mathematical Programming*, 197(2):793–812, 2023.
- Leon Eifler and Ambros Gleixner. Safe and verified gomory mixed-integer cuts in a rational mixed-integer program framework. *SIAM Journal on Optimization*, 34(1):742–763, 2024.
- Jean-Christophe Filliâtre and Andrei Paskevich. Why3—where programs meet provers. In *European symposium on programming*, pages 125–128. Springer, 2013.
- Stephan Gocht, Ciaran McCreesh, Magnus O Myreen, Jakob Nordström, Andy Oertel, and Yong Kiam Tan. End-to-end verification for subgraph solving. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 8038–8047, 2024.
- Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.
- Stefan Szeider. Lrat-catcher: Importing sat solver certificates into lean4 by reflection. *arXiv preprint arXiv:2607.00815*, 2026a.
- Stefan Szeider. Pblean: Pseudo-boolean proof certificates for lean 4. *arXiv preprint arXiv:2602.08692*, 2026b.
- Kenan Wood, Runtian Zhou, Haoze Wu, Hammurabi Mendes, and Jonad Pulaj. Satisfiability modulo theories for verifying milp certificates. *Journal of Symbolic Computation*, page 102543, 2025.